SOFTWARE ENGINEERING · MINI PROJECT

ReturnLoop

Software requirements & design document

SYSTEM	Returns and reverse-logistics centre for an e-commerce business
TEAM	Chirag Seervi, Anshuman Singh, Aryan Singh, Aadi Srivastava
INSTITUTE	SVKM's NMIMS, Hyderabad
VERSION	1.0 · October 2026
LIVE APP	return-loop-c25e.vercel.app



CONTENTS

1. Introduction
2. Functional requirements
3. Non-functional requirements
4. Analysis & design diagrams
    a. Use case diagram
    b. Activity diagram
    c. Swimlane diagram
    d. Class diagram
    e. CRC cards
    f. Data flow diagrams (level 0, level 1)
    g. State diagram
    h. Sequence diagrams
5. UI design: wireframes & prototypes

01 Introduction

1.1 Purpose

ReturnLoop gives an online store one place to run every return: from the customer's request and the doorstep pickup, through quality check and grading, to a correct refund and the item going back on the shelf, to the repair bench or to scrap. Every step is recorded.

1.2 Scope

IN SCOPE

- Six roles and a 13-status return workflow
- Eligibility rules, pickup booking with two attempts
- QC checklist, A / B / C grading, fraud check
- Refurbishing and re-inspection
- Refund calculation and inventory log
- Admin dashboard and frequent-returner flags

OUT OF SCOPE

- Real payments or a payment gateway
- Passwords and real accounts (a role switcher stands in)
- Courier API integration

1.3 Users and roles

ROLE	DEMO USER	MAIN TASKS
Customer	Chirag	Requests returns, books pickups, tracks and closes them
Pickup agent	Ayush Dubey	Collects parcels, or marks the customer absent
Warehouse	Ansuman Singh	Checks parcels in at the returns centre
QC inspector	Aadi Srivastava	Inspects, grades A / B / C, rejects mismatched items
Refurbisher	Anumeet	Repairs grade B items for a second check
Admin	Aryan Singh	Watches totals, reasons, grades and flagged customers

1.4 Key terms

Return code
The ID of a return, e.g. `RL-2026-0004`.

Grade A / B / C
QC's verdict: A restocked (100%), B repaired then rechecked (70%), C scrapped (0%).

Handoff
After each step the app names the next desk and opens its page in one click.

Return window
7 days from delivery; outside it, the item can't be returned.

02 Functional requirements

What the system must do. Priority uses MoSCoW: **Must** is needed for a working return flow, **Should** makes it complete.

ID	REQUIREMENT	DESCRIPTION	ACTOR	PRIORITY
FR01	**Role switch**	Choose any of 6 roles and a user without a password; each page rejects the wrong role (403).	All	MUST
FR02	**View orders**	Customer sees delivered items with eligibility, or the reason it can't be returned.	Customer	MUST
FR03	**Eligibility**	Return allowed within 7 days of delivery, for returnable products, with no other open return.	System	MUST
FR04	**Request return**	Reason, comment (≤ 300 chars), optional photo (PNG/JPG/WEBP, ≤ 3 MB).	Customer	MUST
FR05	**Book pickup**	Date from tomorrow to +5 days, slot Morning / Afternoon / Evening; reschedule.	Customer	MUST
FR06	**Collect parcel**	Agent marks picked up or absent; the 2nd absence cancels the return.	Pickup agent	MUST
FR07	**Receive**	Warehouse checks the parcel in; it joins the QC queue.	Warehouse	MUST
FR08	**Inspect & grade**	QC checklist, suggested grade, A/B/C; a mismatched item is rejected as fraud.	QC inspector	MUST
FR09	**Refurbish**	Grade B goes to repair, then re-inspection: pass → restock (70%), fail → scrap.	Refurbisher, QC	MUST
FR10	**Refund**	final = max(0, base × grade% − pickup fee), damaged/wrong-item exception, created once.	System	MUST
FR11	**Track**	Live status (polled every 5 s), history timeline and refund breakdown.	Customer	SHOULD
FR12	**Dashboard**	Totals, reasons and grades charts, frequent-returner flags (> 3 in 30 days).	Admin	SHOULD
FR13	**Hand off**	After each step, a card names the next desk and opens its page in one click.	All	SHOULD

03 Non-functional requirements

How well the system must do it, and how each quality is checked.

ID	QUALITY	REQUIREMENT	HOW IT IS CHECKED
NFR01	**Performance**	60 fps on a mid laptop, 30+ on phones; pixel ratio capped; 3D pauses off-screen.	Frame-rate guard halves particles below 40 fps
NFR02	**Reliability**	Refund + inventory log + status change in one transaction; no double refunds.	Unique refunds.return_id; double-submit tests
NFR03	**Security**	Role guard on every blueprint; customers only see their own returns; upload validation.	403/404 tests; file-signature check on photos
NFR04	**Usability**	Plain-language errors and empty states; the handoff card always names the next step.	End-to-end relay test over HTTP
NFR05	**Accessibility**	WCAG AA contrast, keyboard-only use, visible focus, reduced-motion mode, text on every badge.	Manual keyboard pass; reduced-motion mode
NFR06	**Maintainability**	Logic in services/, rules in config.py, thin routes; a pytest suite over services, pages and the full flow.	pytest on every increment
NFR07	**Portability**	Python 3.10+, SQLite, no build step: install, seed, run in 3 commands.	Runs locally and on Vercel
NFR08	**Robustness**	Static fallbacks without WebGL or internet; friendly 403 / 404 / 413 pages.	Pages tested with 3D unavailable

04 Analysis & design diagrams

Each diagram below is drawn from the code: the statuses and moves come from `services/state_machine.py`, the classes from `models.py`, and the messages from the real functions.

4.1 Use case diagram

Six actors and what each can do. Requesting a return always includes the eligibility check; grading includes issuing the refund.

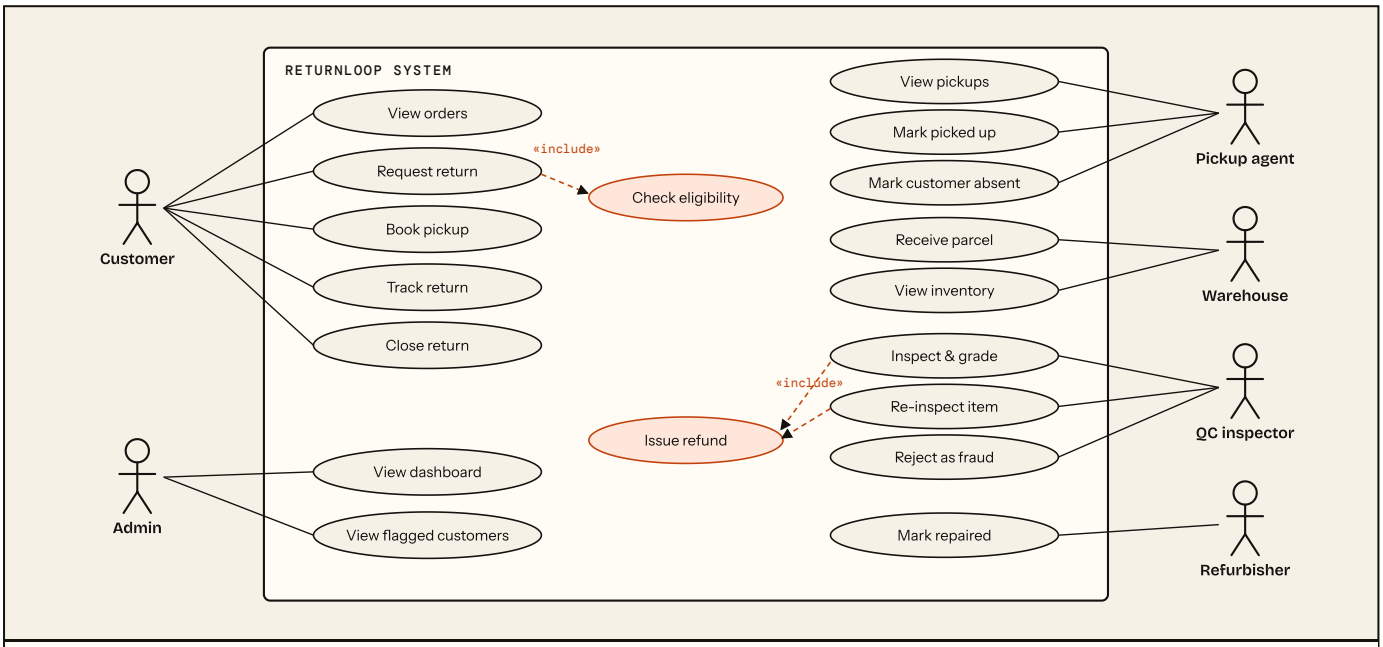


FIGURE 4.1 Use case diagram

4.2 Activity diagram

One return from the customer's orders to the refund, with every decision: eligibility, whether the customer is home, whether the item matches the order, the grade, and the recheck.

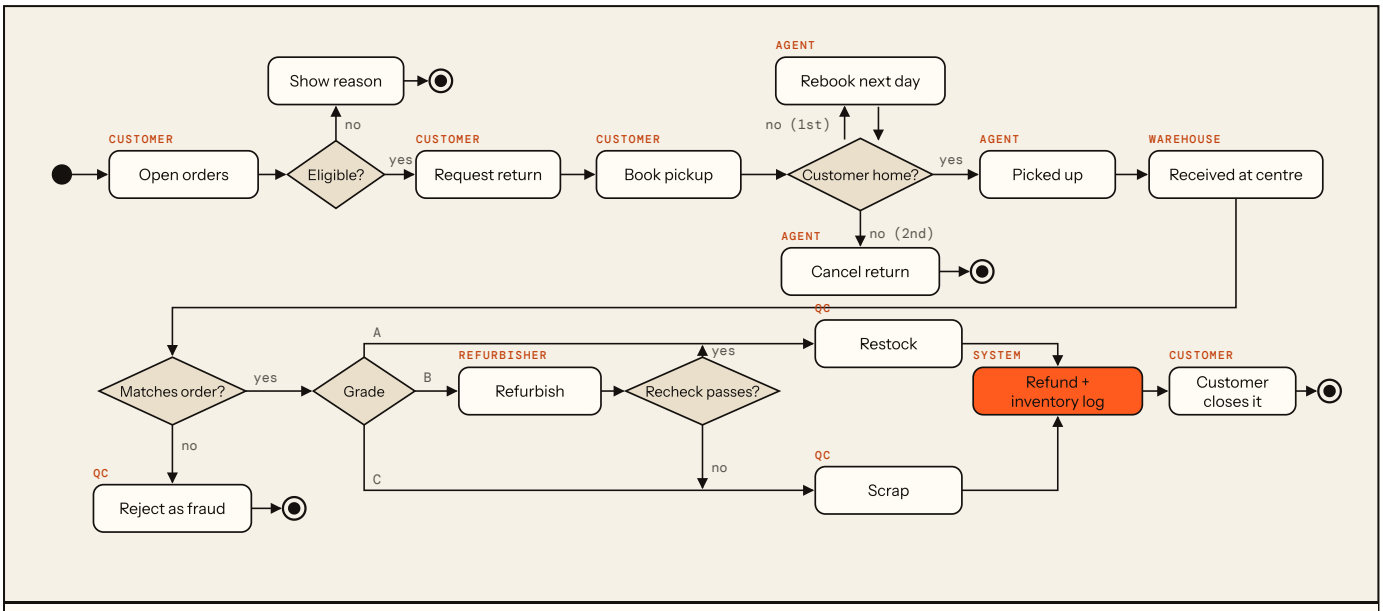


FIGURE 4.2 Activity diagram: one return, start to finish

4.3 Swimlane diagram

The same flow split by who acts: each column is a role, so every arrow that crosses a column is a handoff from one desk to the next.

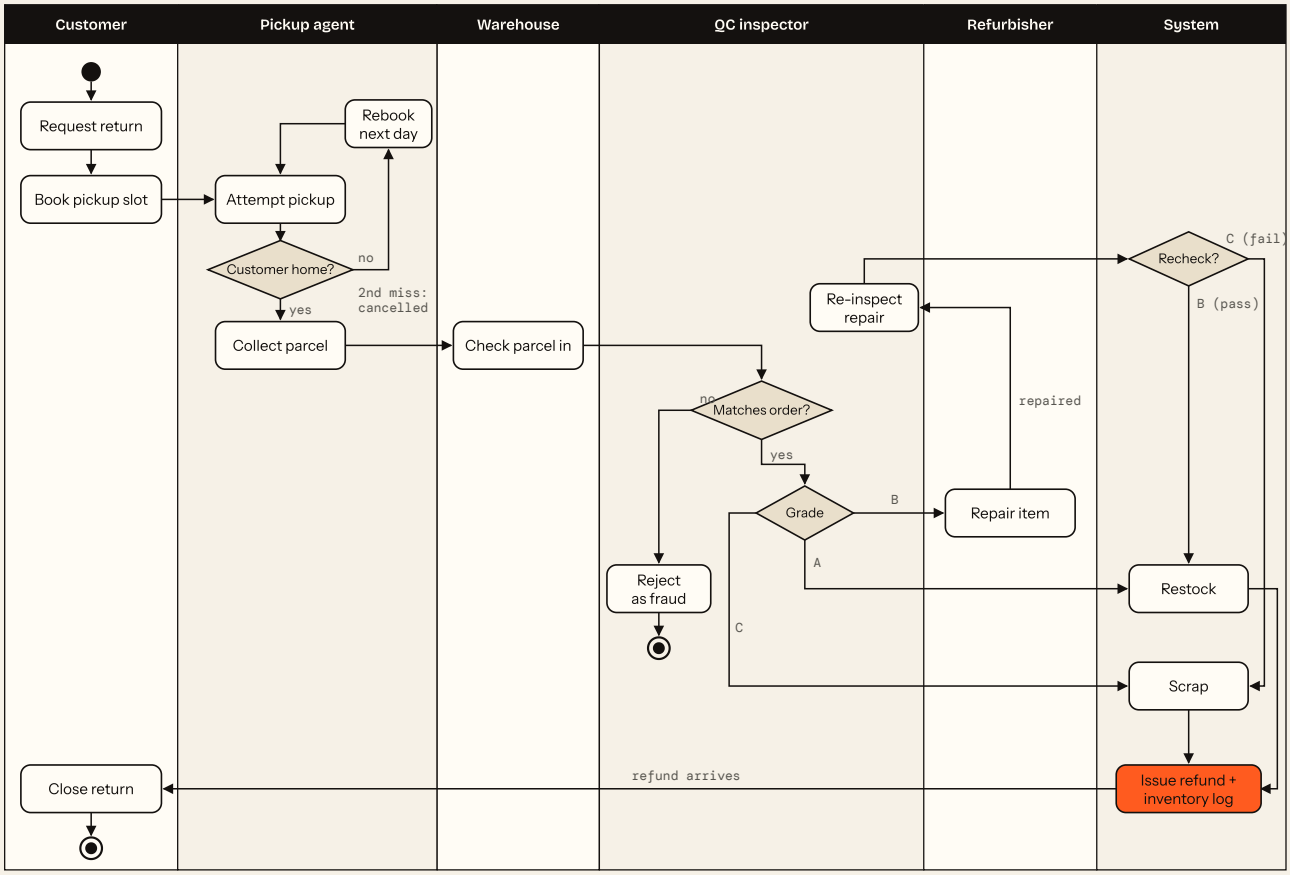


FIGURE 4.3 Swimlane activity diagram

4.4 Class diagram

The domain model in `models.py`: eleven classes, one per database table. A return request owns its pickups, QC reports, repair jobs, refund and history.

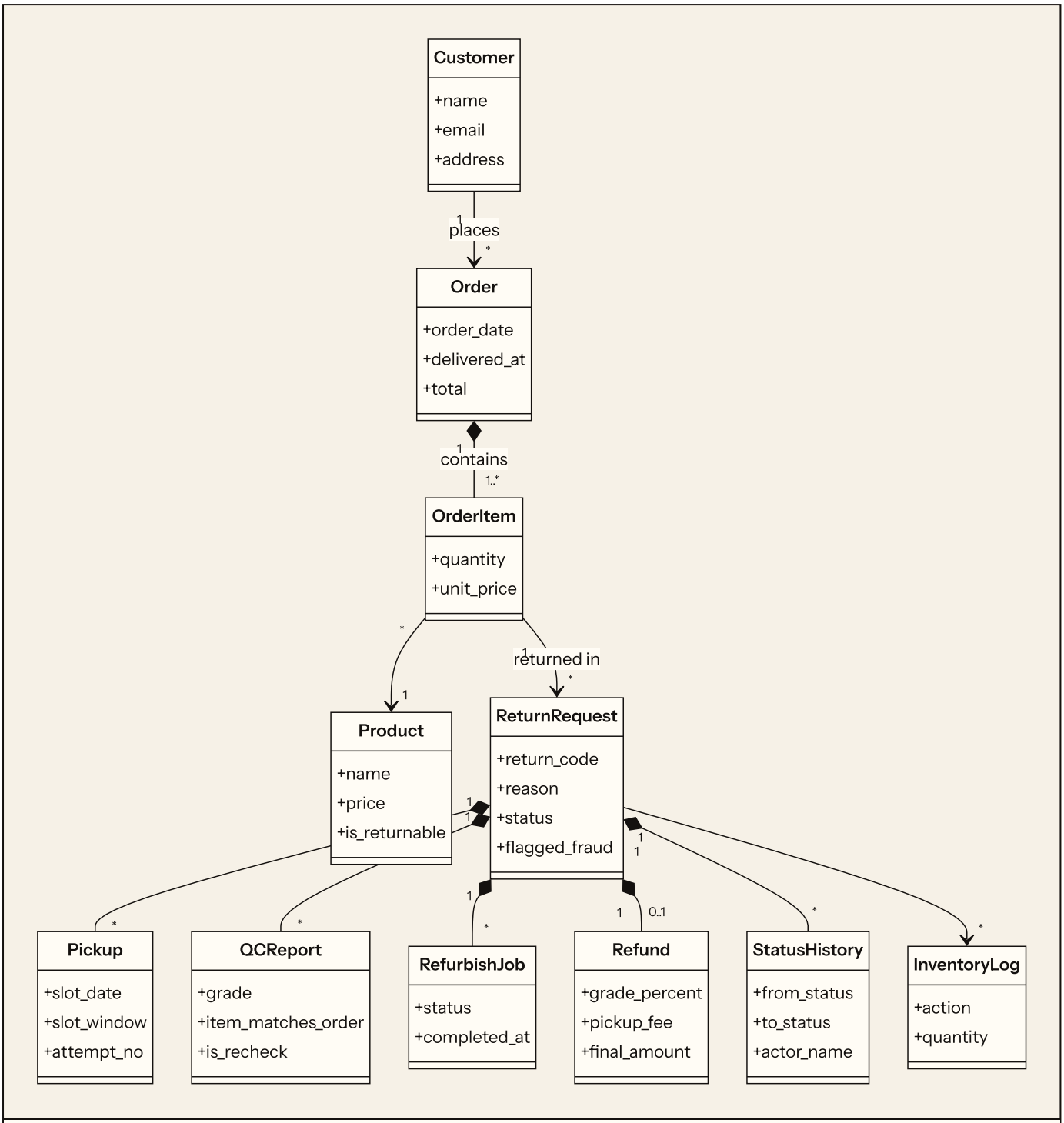


FIGURE 4.4 Class diagram

4.5 CRC cards

Class, responsibilities, collaborators: what each key class (or service) knows and does, and whom it works with.

ReturnRequest

models.py

RESPONSIBILITIES

- Holds reason, comment, photo and status
- Knows its pickups, QC reports, refund and history
- Gives its current pickup and latest report

COLLABORATORS

- OrderItem
- Pickup
- QCReport
- Refund
- StatusHistory

State machine

services/state_machine.py

RESPONSIBILITIES

- Allows only the moves in the transition table
- Writes a history row for every move
- Refuses anything else

COLLABORATORS

- ReturnRequest
- StatusHistory

Eligibility

services/eligibility.py

RESPONSIBILITIES

- Checks the 7-day window
- Checks the product is returnable
- Checks there is no open return

COLLABORATORS

- OrderItem
- Product
- ReturnRequest

Pickup

models.py

RESPONSIBILITIES

- Holds slot date and window
- Counts attempts (max 2)
- Records the outcome

COLLABORATORS

- ReturnRequest
- Pickup agent

QCReport

models.py

RESPONSIBILITIES

- Holds the checklist and grade
- Marks a recheck
- Keeps the inspector's remarks

COLLABORATORS

- ReturnRequest
- QC inspector

RefurbishJob

models.py

RESPONSIBILITIES

- Tracks open / repaired
- Keeps repair notes and technician

COLLABORATORS

- ReturnRequest
- Refurbisher

Refund service

services/refund.py

RESPONSIBILITIES

- Computes base × grade% − fee
- Applies the damaged / wrong-item exception
- Creates the refund and inventory log once

COLLABORATORS

- ReturnRequest
- QCReport
- Refund
- InventoryLog
- State machine

Handoff

services/handoff.py

RESPONSIBILITIES

- Works out who acts next
- Picks the page they land on
- Lists returns each desk handed on

COLLABORATORS

- ReturnRequest
- Pickup
- RefurbishJob

Flags	services/flags.py
RESPONSIBILITIES • Counts returns per customer in 30 days • Flags more than 3	COLLABORATORS • Customer • ReturnRequest

4.6 Data flow diagrams

Level 0 shows the whole system as one process and the data each role sends and receives. **Level 1** opens it into eight processes and six data stores.

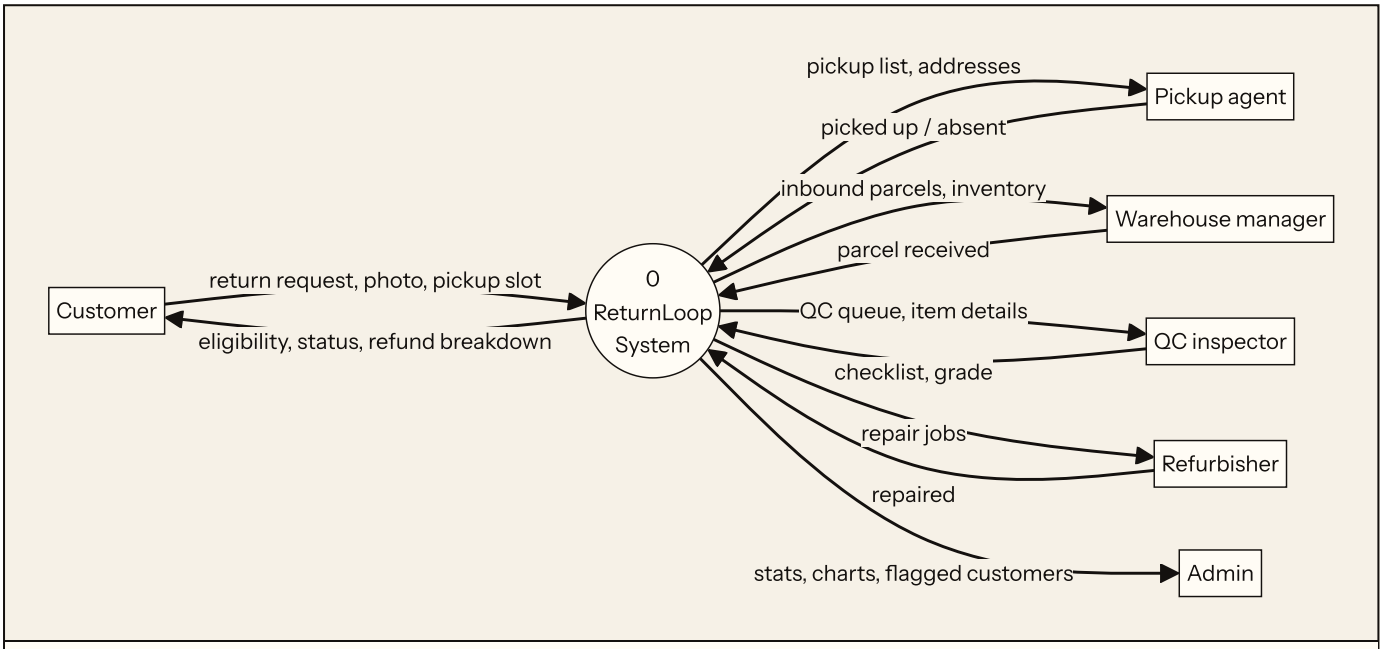


FIGURE 4.6a DFD level 0 (context diagram)

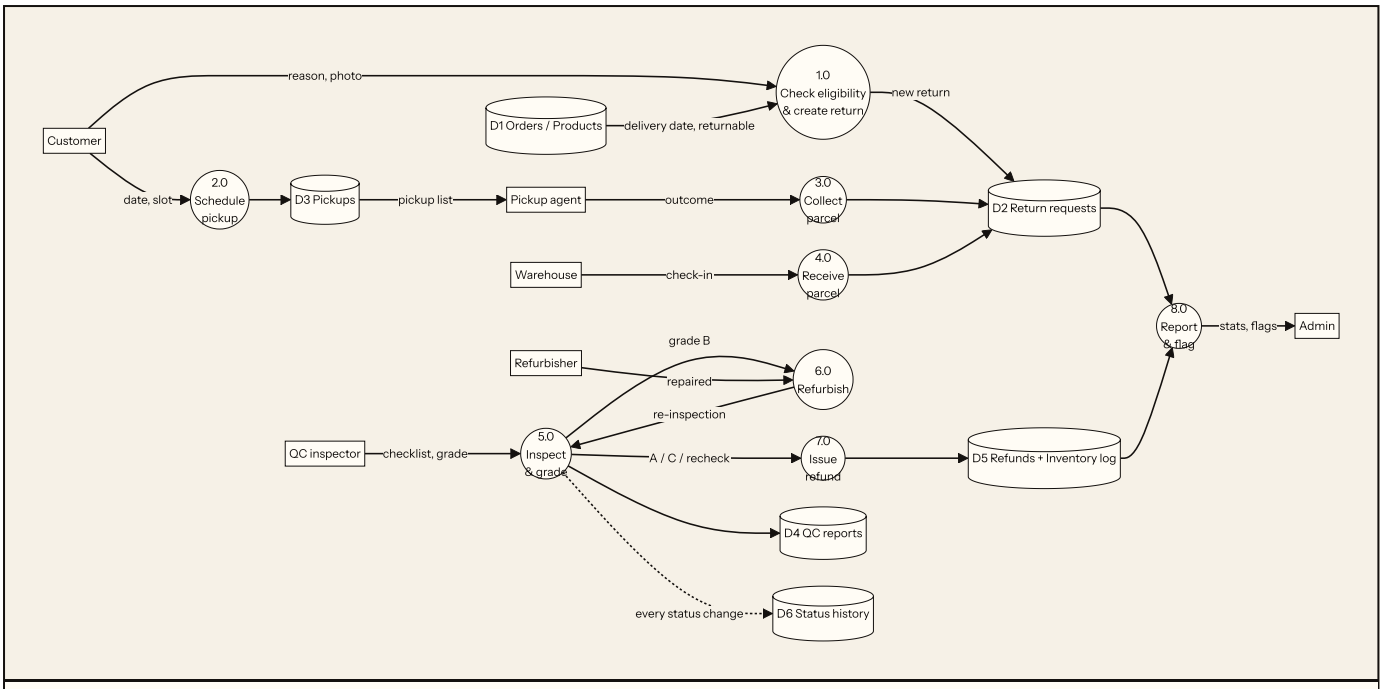


FIGURE 4.6b DFD level 1: processes and data stores

4.7 State diagram

All 13 statuses of a return. Boxes list the actions on entering (`entry/`) and while in the state (`do/`); arrows read `event [guard] / action` . `change_status()` refuses any move not drawn here.

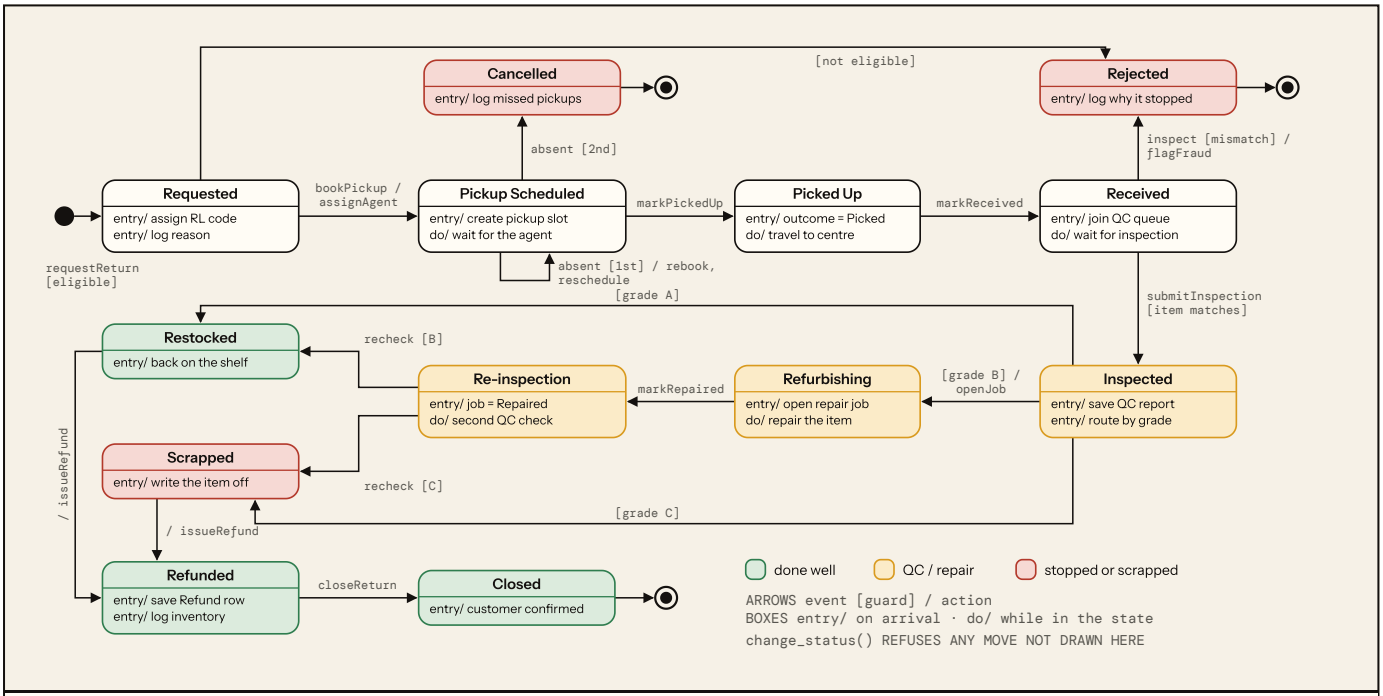


FIGURE 4.7 State diagram of a return

4.8 Sequence diagrams

(a) QC grades a return A: the inspection, two status changes and the refund are committed in one transaction. **(b)** The handoff: after the agent collects a parcel, the next page shows a card that switches to the warehouse in one click.

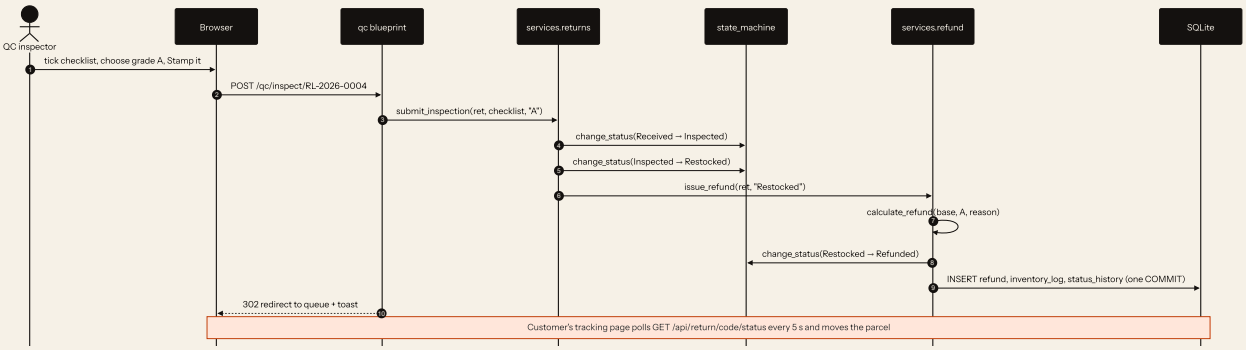


FIGURE 4.8a Sequence: QC grades a return and the refund is issued

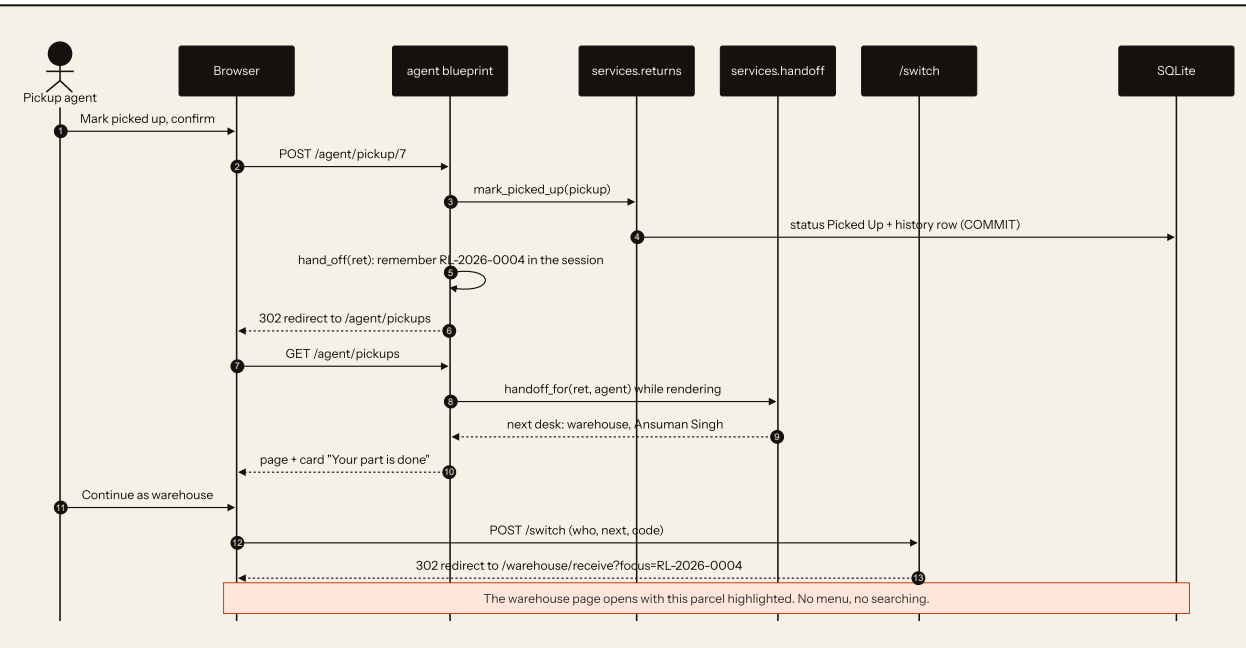


FIGURE 4.8b Sequence: handing a return to the next desk

05 UI design: wireframes & prototypes

5.1 Design language

"Mailroom meets control tower": kraft paper, ink and one signal orange. Return codes and labels are set in a monospace font like shipping labels; grades are rubber stamps; every status badge carries text, so colour is never the only signal.

Paper	Kraft	Ink	Signal orange	Grade A
#F6F1E7	#E9DFCB	#14110F	#FF5B1F	#2F7D4F

Grade B	Grade C
#D99A1E	#B3392B

Bricolage Grotesque headings · **Instrument Sans** body · DM Mono codes & labels

5.2 Wireframes

Low-fidelity sketches of the main screens, drawn before the visual design: layout, content and the one main action on each screen.



FIGURE 5.1 Customer · My orders

Each delivered item is a shipping slip with its return window and one button.



FIGURE 5.2 Customer · Request return

The item on the left; reason tiles, comment and an optional photo on the right.

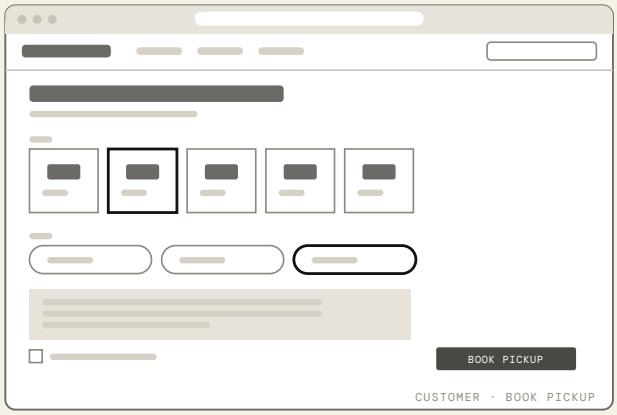


FIGURE 5.3 Customer · Book pickup

Five day tiles, three slots, the pickup address to confirm.

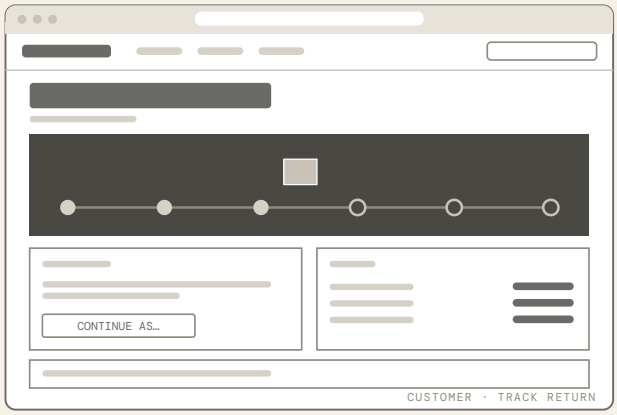


FIGURE 5.4 Customer · Track return

Six-step progress, what happens next, refund breakdown, history.

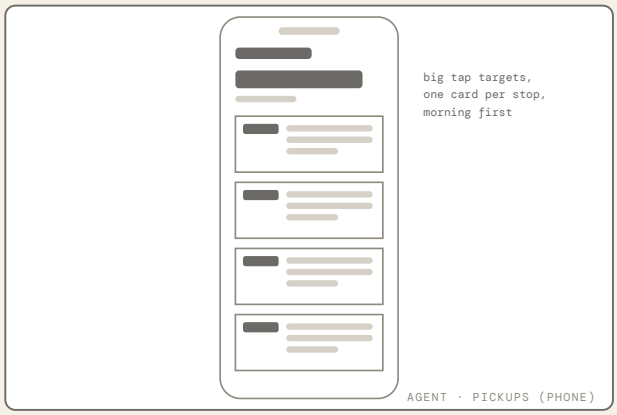


FIGURE 5.5 Agent · Pickups (phone)

Phone-first list of stops, morning first, large tap targets.

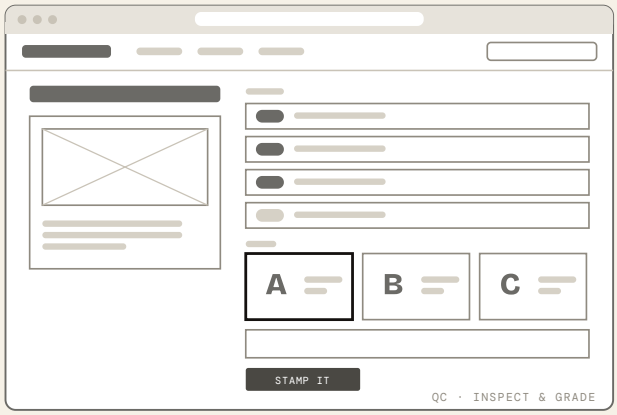


FIGURE 5.6 QC · Inspect & grade

Checklist toggles suggest a grade; the inspector stamps A, B or C.

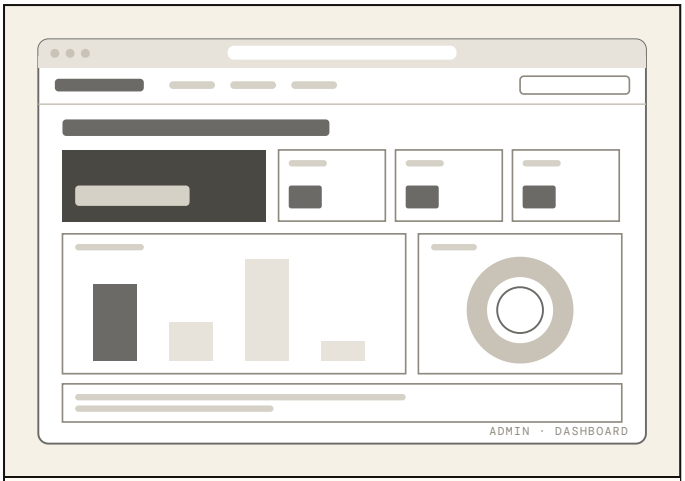


FIGURE 5.7 Admin · Dashboard

Key numbers on top, reasons and grades charts, flagged customers.

5.3 Prototypes

High-fidelity screens of the working prototype, the app as built. Each follows its wireframe above.

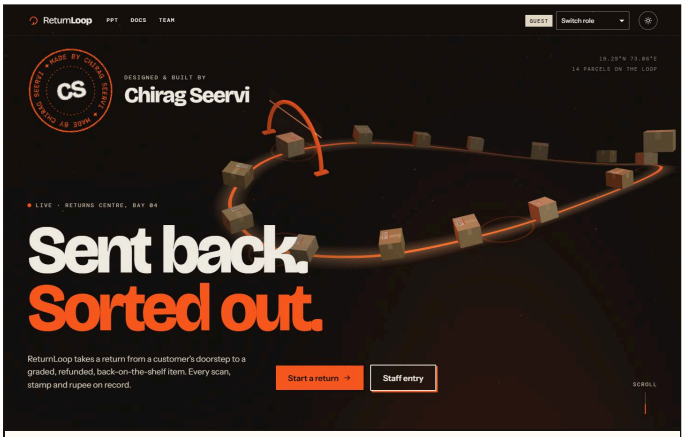


FIGURE 5.8 Landing page

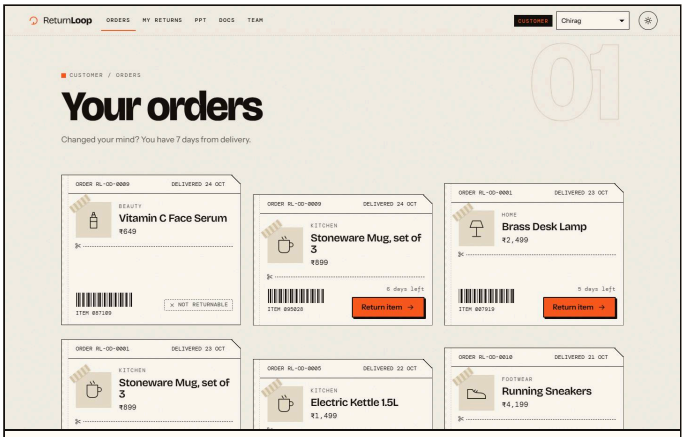


FIGURE 5.9 Customer · My orders

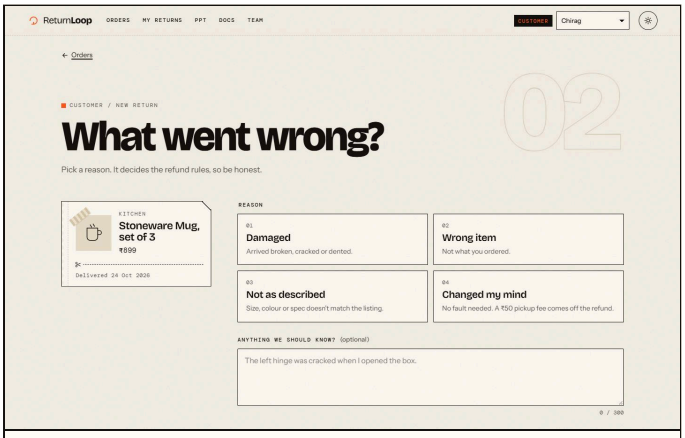


FIGURE 5.10 Customer · Request return

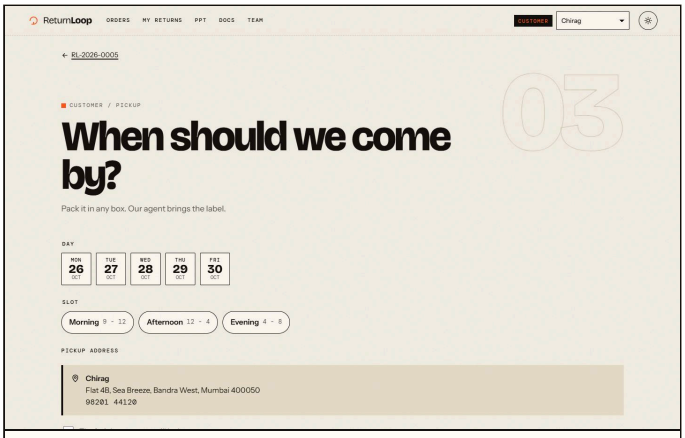


FIGURE 5.11 Customer · Book pickup

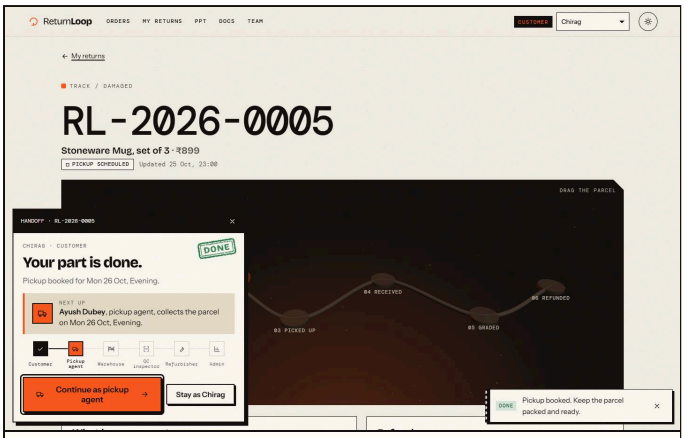


FIGURE 5.12

Customer · Track return, with the handoff card

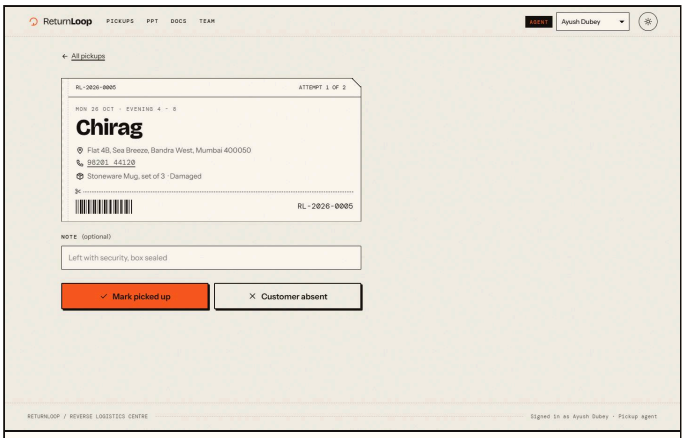


FIGURE 5.13 Agent · Pickup slip

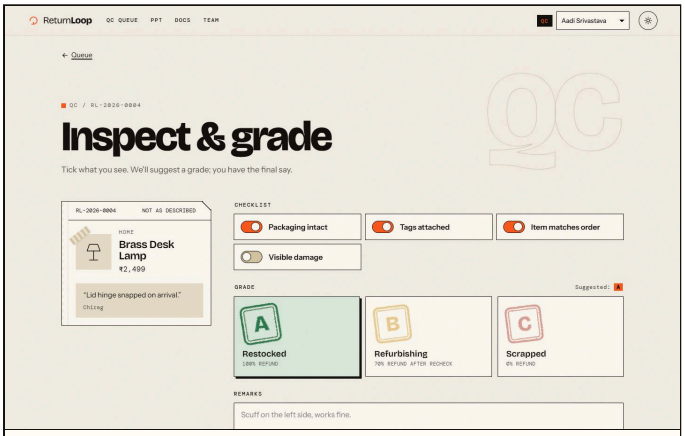


FIGURE 5.14 QC · Inspect & grade

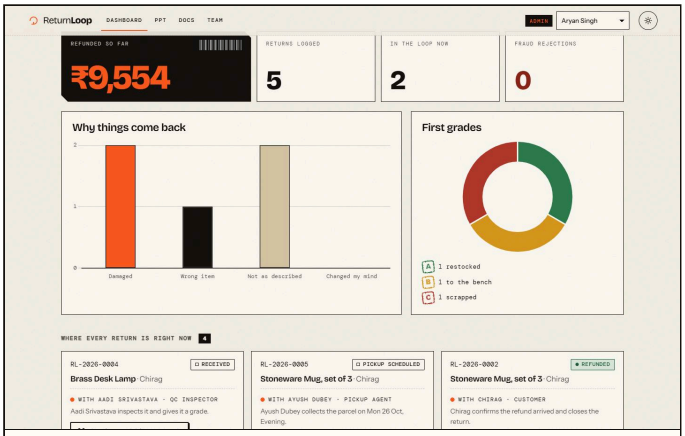


FIGURE 5.15 Admin · Dashboard

RETURNLOOP · SOFTWARE REQUIREMENTS & DESIGN · v1.0

Chirag Seervi · Anshuman Singh · Aryan Singh · Aadi Srivastava